Overview
Welcome to Semantica

Core Concepts
Core Concepts
Modules
Choose the Right Module
Glossary

Guides
Architecture
Learning More
GraphRAG: Graph-Augmented Retrieval
Ingest Anything
LLM Integrations
Pipeline Builder
Multi-Agent Systems
Decision Intelligence
Context Graphs
Agent Memory
Provenance & Audit Trails
Semantic Extraction
Ontology Management
Reasoning & Rules
Export & Serialization
Deduplication & Entity Merging
MCP Server
SHACL Validation
Conflict Detection & Resolution
Change Management & Versioning
Policy Engine
Visualization
Distance Intelligence
Graph Analytics

Integrations
Agno Integration
CrewAI Integration
LangChain Integration
Docling Integration
Snowflake Integration
Databricks Integration
Salesforce Integration

Graph Stores
Apache AGE Graph Store

Vector Stores
pgvector Store

FAQ
FAQ

Community
Community
Community Projects
Contributing
Governance
Citation
License

Guides

Reasoning & Rules

Apply forward-chaining, backward-chaining, Datalog, SPARQL, RETE, temporal interval, and LLM-based reasoning over your knowledge graph — derive new facts, check constraints, and explain inferences.

On this page
What Is Reasoning?
Why Use Reasoning?
When To Use / When Not To Use
Key Reasoning Concepts
How Graph Data Becomes Reasoning Facts
Choosing a reasoning mode
Step 1 — Ground facts and working memory
Step 2 — Forward chaining: materialising derived facts
Step 3 — Backward chaining: proving a specific goal
Step 4 — Recursive inference with Datalog
Step 5 — SPARQL queries over enriched working memory
Step 6 — RETE engine for large rule sets
Step 7 — Temporal interval reasoning
Step 8 — LLM-based graph reasoning
Step 9 — Explaining inferences in natural language
Putting it together: a complete reasoning pipeline
Domain examples
Common Pitfalls
Related Guides

Semantica's reasoning layer encodes domain logic as rules and applies it to your knowledge graph to derive conclusions that no single document states explicitly. Eight complementary reasoning modes — from symbol-rule forward chaining to recursive Datalog and LLM-backed freeform queries — let you choose the right tool for each inference problem without switching frameworks.

What Is Reasoning?

Reasoning derives implicit knowledge from explicit facts using logical rules. Unlike retrieval, search, or graph traversal, reasoning generates new conclusions that weren't directly stated in your original data.

Reasoning vs. Retrieval: Retrieval finds existing information that matches your query. Reasoning applies logical rules to derive new facts that weren't explicitly stored.

Reasoning vs. Search: Search matches keywords or semantic similarity. Reasoning uses logical inference to conclude facts like "if A implies B, and A is true, then B must be true."

Reasoning vs. Graph Traversal: Traversal follows existing edges between nodes. Reasoning can infer new relationships based on rules — for example, concluding that two entities are related even if no direct edge exists.

Why Use Reasoning?

Deriving implicit knowledge. Documents might state "APT29 uses SUNBURST" and "SUNBURST exploits CVE-2020-10148" separately, but reasoning concludes "APT29 exploits CVE-2020-10148" automatically.

Identifying patterns. Rules can detect complex patterns across your knowledge graph — threat actors that share TTPs, suppliers in transitive relationships, or compliance violations that emerge from combinations of conditions.

Supporting investigations. Reasoning helps analysts by highlighting non-obvious connections, flagging entities that meet risk criteria, and explaining why conclusions were reached.

Decision support. Encode policies, regulations, or business rules as logical statements. Reasoning engines evaluate them consistently and provide audit trails for decisions.

When To Use / When Not To Use

Use reasoning for:

- Complex domains with well-defined logical relationships
- Policy enforcement and compliance checking
- Multi-step inference where conclusions depend on chains of facts
- Situations requiring explainable decisions with audit trails
- Identifying implicit relationships that aren't explicitly stated

Retrieval may be sufficient for:

- Finding documents or information that directly answers your question
- Exploratory research where you don't know what patterns to look for
- Simple keyword or semantic searches

Graph traversal may be sufficient for:

- Following explicit relationships between entities
- Neighborhood analysis around known starting points
- Path-finding between entities with direct connections

Reasoning provides additional value when:

- Your domain has logical rules that can infer new facts
- You need to detect patterns that require multiple conditions
- Decisions must be explainable and auditable
- Implicit relationships are as important as explicit ones

Key Reasoning Concepts

Datalog is a logic programming language based on rules and facts. Facts are simple statements like "parent(tom, bob)". Rules derive new facts: "grandparent(X, Z) :- parent(X, Y), parent(Y, Z)". Datalog excels at recursive queries and transitive relationships.

SPARQL is a query language for RDF data that can incorporate inference rules. It uses triple patterns to match graph data and can be extended with reasoning to derive implicit triples before querying.

RETE is an algorithm for efficiently evaluating many rules against a working memory of facts. It builds a network that avoids re-evaluating unchanged conditions, making it suitable for systems with hundreds of rules or streaming data.

Rule-based inference applies "if-then" rules to derive new conclusions. Forward chaining applies all applicable rules to derive everything possible. Backward chaining works backwards from a goal to find the minimal proof.

How Graph Data Becomes Reasoning Facts

When using `DatalogReasoner.load_from_graph()`, your knowledge graph's nodes and edges are converted into Datalog facts. All predicates and arguments are lowercased:

- A node with type "ThreatActor" and id "APT29" becomes `threatactor(apt29)`
- An edge from APT29 to SUNBURST with type "uses" becomes `uses(apt29, sunburst)`

The reasoning engine treats these facts as the starting point for inference, applying rules to derive new conclusions that get added to working memory.

> ⓘ The reasoning module operates over facts you supply directly or load from a `ContextGraph`. Derived facts are added to working memory and are immediately available for further inference in the same session. To persist derived facts back into the graph, pass them to `AgentContext.store()`.

Choosing a reasoning mode

MODE	BEST FOR	CLASS
Forward chaining	Materialise all implied facts from ground truth	`Reasoner.forward_chain()`
Backward chaining	Prove a specific goal; get the minimal evidence chain	`Reasoner.backward_chain()`
Datalog	Recursive traversal of arbitrary depth (supply chains, org graphs)	`DatalogReasoner`
SPARQL	Pattern-matching queries over enriched working memory	`SPARQLReasoner`
RETE	100+ rule sets — incremental fact propagation via alpha/beta network	`ReteEngine`
Temporal	Allen interval relations between time windows	`TemporalReasoningEngine`
Natural language	LLM-backed freeform queries over graph context	`GraphReasoner`
Explanation	Translate inference results into human-readable justifications	`ExplanationGenerator`

Step 1 — Ground facts and working memory

The `Reasoner` class maintains a set of ground facts and a list of rules. Facts can be added as predicate strings or loaded from a `ContextGraph`. Start with the explicit knowledge your extraction pipeline produced:

```
from semantica.reasoning import Reasoner, Rule, RuleType

reasoner = Reasoner()

# Ground facts: Predicate(arg) or Predicate(arg1, arg2)
reasoner.add_fact("ThreatActor(APT29)")
reasoner.add_fact("ThreatActor(GAMMA-7)")
reasoner.add_fact("ThreatActor(DELTA-3)")
reasoner.add_fact("Exploits(APT29, CVE-2025-3400)")
reasoner.add_fact("Exploits(GAMMA-7, CVE-2025-1234)")
reasoner.add_fact("Exploits(GAMMA-7, CVE-2025-5678)")
reasoner.add_fact("CriticalVuln(CVE-2025-3400)")
reasoner.add_fact("CriticalVuln(CVE-2025-1234)")
reasoner.add_fact("CriticalVuln(CVE-2025-5678)")
reasoner.add_fact("Targets(APT29, NATOLogistics)")
reasoner.add_fact("Targets(GAMMA-7, NATOLogistics)")
reasoner.add_fact("SuppliedExploits(DELTA-3, GAMMA-7)")
reasoner.add_fact("SectorOverlap(NATOLogistics, CriticalInfrastructure)")
```

These ground facts represent what documents explicitly stated. The rules you add next tell the system what those facts *imply*.

Step 2 — Forward chaining: materialising derived facts

Forward chaining starts from ground facts and applies every matching rule until no new conclusions can be drawn — reaching fixpoint:

```
# String-format rules are parsed automatically
# Variables are single uppercase letters or multi-character uppercase words
reasoner.add_rule(
    "IF ThreatActor(X) AND Exploits(X, Y) AND CriticalVuln(Y) THEN HighRiskActo
)
reasoner.add_rule(
    "IF HighRiskActor(X) AND Targets(X, Z) THEN CriticalTarget(Z)"
)
reasoner.add_rule(
    "IF SuppliedExploits(A, B) AND HighRiskActor(B) THEN HighRiskSupplier(A)"
)

# forward_chain() applies all rules until fixpoint
derived = reasoner.forward_chain()

for result in derived:
    print("{:<40s}  conf={:.0%}  rule={}".format(
        result.conclusion,
        result.confidence,
        result.rule_used.name if result.rule_used else "n/a",
    ))
```

```
HighRiskActor(APT29)                      conf=100%  rule=Rule 1
HighRiskActor(GAMMA-7)                    conf=100%  rule=Rule 1
CriticalTarget(NATOLogistics)             conf=100%  rule=Rule 2
HighRiskSupplier(DELTA-3)                 conf=100%  rule=Rule 3
```

DELTA-3 is flagged even though no document described it that way — the system traced: DELTA-3 supplied GAMMA-7, and GAMMA-7 exploits critical CVEs. For rules that need priority ordering or graded confidence, use the `Rule` dataclass:

If a rule has side-effecting actions, one concrete activation runs those actions at most once on a Reasoner instance. Re-running `forward_chain()` is therefore safe: already-attempted actions are not repeated. Use `reasoner.reset_action_history()` when you intentionally want to replay them; `reasoner.clear()` and `reasoner.reset()` also clear the history.

```
# Higher priority rules fire first; confidence propagates into Infe
reasoner.add_rule(Rule(
    rule_id="attr-1",
    name="ttp_match_attribution",
    conditions=["ThreatActor(X)", "Exploits(X, CVE)", "CriticalVuln(CVE)"],
    conclusion="HighRiskActor(X)",
    rule_type=RuleType.IMPLICATION,
    confidence=0.92,
    priority=10,
))

reasoner.add_rule(Rule(
    rule_id="attr-2",
    name="supplier_elevation",
    conditions=["SuppliedExploits(A, B)", "HighRiskActor(B)"],
    conclusion="HighRiskSupplier(A)",
    rule_type=RuleType.IMPLICATION,
    confidence=0.85,
    priority=5,
))
```

Step 3 — Backward chaining: proving a specific goal

Backward chaining tests a single hypothesis by working backward through rules — the right tool when you need a yes/no answer and the minimal evidence chain without deriving every other possible fact first:

```
# backward_chain() returns an InferenceResult if the goal is provab
result = reasoner.backward_chain("HighRiskSupplier(DELTA-3)", max_depth=5)

if result:
    print("Proved: {}".format(result.conclusion))
    print("Via premises:")
    for p in result.premises:
        print("  - {}".format(p))
    print("Confidence: {:.0%}".format(result.confidence))
else:
    print("Goal not provable — DELTA-3 is not classified as a high-risk supplie
          "given current facts and rules")
```

```
Proved: HighRiskSupplier(DELTA-3)
Via premises:
  - SuppliedExploits(DELTA-3, GAMMA-7)
  - HighRiskActor(GAMMA-7)
Confidence: 85%
```

The premises list is the explanation chain — each item is a fact that was necessary to reach the conclusion. Show this to analysts when they ask "why is DELTA-3 classified as high-risk?"

Step 4 — Recursive inference with Datalog

`DatalogReasoner` handles questions requiring arbitrary-depth traversal — "which actors can transitively reach critical infrastructure?" — using recursive Horn clause rules with semi-naive bottom-up fixpoint evaluation:

```
from semantica.reasoning import DatalogReasoner

dl = DatalogReasoner()

# EDB (Extensional DB) — base facts; arguments are constants (lowercase)
dl.add_fact("supplied(delta3, gamma7)")
dl.add_fact("supplied(gamma7, apt29_affiliate)")
dl.add_fact("supplied(apt29_affiliate, apt29)")
dl.add_fact("targets(apt29, nato_logistics)")
dl.add_fact("targets(gamma7, nato_logistics)")
dl.add_fact("sector(nato_logistics, critical_infrastructure)")

# IDB (Intensional DB) — recursive rules; uppercase = variable
# Base case: direct supply link
dl.add_rule("reaches(X, Y) :- supplied(X, Y).")
# Recursive case: X reaches Y if X supplies Z and Z reaches Y
dl.add_rule("reaches(X, Y) :- supplied(X, Z), reaches(Z, Y).")

# Derived predicate combining reachability with sector membership
dl.add_rule("sector_exposure(Actor, Sector) :- reaches(Actor, Target), sector(T
dl.add_rule("sector_exposure(Actor, Sector) :- targets(Actor, Target), sector(T

# derive_all() runs semi-naive fixpoint until no new facts emerge
dl.derive_all()

# Query with free variables — returns list of binding dicts
exposures = dl.query("sector_exposure(?actor, critical_infrastructure)")
for row in exposures:
    print("CI-exposed actor: {}".format(row["actor"]))
```

```
CI-exposed actor: delta3
CI-exposed actor: gamma7
CI-exposed actor: apt29_affiliate
CI-exposed actor: apt29
```

DELTA-3 appears even though no document connects it directly to critical infrastructure. Datalog traced the full chain: delta3 → gamma7 → apt29_affiliate → apt29 → nato_logistics → critical_infrastructure.

Bind variables to ask directed questions:

```
# Which sectors does delta3 have exposure to?
rows = dl.query("sector_exposure(delta3, ?sector)")
for row in rows:
    print("delta3 → sector: {}".format(row["sector"]))
```

Skip manual `add_fact()` calls by loading a `ContextGraph` directly:

```
from semantica.context import ContextGraph

graph = ContextGraph()
# ... graph populated by AgentContext.store() or extraction pipeline ...
count = dl.load_from_graph(graph)
print("Loaded {} facts from graph".format(count))
```

Step 5 — SPARQL queries over enriched working memory

After forward chaining has derived new facts, `SPARQLReasoner` prepares SPARQL queries over the enriched working memory with optional inference expansion:

```
from semantica.reasoning import SPARQLReasoner

sparql = SPARQLReasoner()

# Inference rules expand the SPARQL query before execution
sparql.add_inference_rule(
    "IF ThreatActor(X) AND Exploits(X, Y) AND CriticalVuln(Y) THEN HighRiskActo
)

query = """
    SELECT ?actor ?cve WHERE {
        ?actor <Exploits> ?cve .
        ?cve   <CriticalVuln> true .
    }
"""

# expand_query() applies inference rules to the query text:
expanded = sparql.expand_query(query)
print(expanded)
```

`execute_query()` is not implemented yet: no triplet-store execution path exists, so it raises `NotImplementedError` rather than returning an empty result set that callers would misread as “no matches”. Until execution lands, run the expanded query against your RDF store directly (for example with `rdflib`).

Inspect the expanded query before running it:

```
# See the query after inference rules are applied
expanded = sparql.expand_query(query)
print(expanded)
```

Step 6 — RETE engine for large rule sets

`ReteEngine` implements the RETE algorithm — a network of alpha nodes (single-condition matching) and beta nodes (join operations) that avoids re-evaluating unchanged conditions on every new fact. Use it when you have 100+ rules or need incremental fact propagation in a streaming or event-driven setting:

```
from semantica.reasoning import ReteEngine, Rule, RuleType, Fact

# Define rules as Rule objects — same Rule class used by Reasoner
rules = [
    Rule(
        rule_id="r1", name="port_scan_detected",
        conditions=["PortScan(Source)", "HighFrequency(Source)"],
        conclusion="Scanning(Source)",
        confidence=0.90, priority=10,
    ),
    Rule(
        rule_id="r2", name="c2_beacon_identified",
        conditions=["Scanning(Source)", "BeaconPattern(Source, Dest)"],
        conclusion="C2Channel(Source, Dest)",
        confidence=0.85, priority=8,
    ),
    Rule(
        rule_id="r3", name="lateral_movement_detected",
        conditions=["C2Channel(Source, Dest)", "InternalHost(Dest)"],
        conclusion="LateralMovement(Source, Dest)",
        confidence=0.80, priority=5,
    ),
]

engine = ReteEngine()
engine.build_network(rules)   # compile rules into alpha/beta/terminal node net

# Facts are structured: Fact(fact_id, predicate, [arguments])
engine.add_fact(Fact("f1", "PortScan",      ["192.168.1.50"]))
engine.add_fact(Fact("f2", "HighFrequency", ["192.168.1.50"]))
engine.add_fact(Fact("f3", "BeaconPattern", ["192.168.1.50", "10.0.0.5"]))
engine.add_fact(Fact("f4", "InternalHost",  ["10.0.0.5"]))

# match_patterns() returns Match objects: rule + matched facts + confidence
matches = engine.match_patterns()
print("Rule activations: {}".format(len(matches)))

# execute_matches() fires each matched rule and returns the derived conclusions
conclusions = engine.execute_matches(matches)
for c in conclusions:
    print("Derived:", c)

# Network diagnostics — useful for verifying rule compilation
stats = engine.get_network_stats()
print("Nodes: {total_nodes}  Alpha: {alpha_nodes}  Beta: {beta_nodes}  Facts: {

# Clear working memory without recompiling the rule network
engine.reset()
```

The rule network is compiled once by `build_network()`. Each subsequent `add_fact()` call propagates incrementally through only the nodes whose conditions it satisfies — not the full rule set — which keeps evaluation cost proportional to the number of new activations rather than the total rule count.

With a Reasoner bound, Rete action side effects are attempted once per rule, bindings, and matched fact identity. Passing the same match to `execute_matches()` again still returns the same conclusion, but does not repeat its actions. Call `engine.reset_action_history()` to replay actions without clearing working memory. `engine.reset()` and `engine.build_network()` also clear the action history.

Step 7 — Temporal interval reasoning

`TemporalReasoningEngine` computes Allen interval relations between time windows, letting you identify whether two events overlap, one contains the other, they meet at a boundary, and so on across your graph:

```
from datetime import datetime, timezone
from semantica.reasoning import TemporalReasoningEngine, TemporalInterval, Inte

engine = TemporalReasoningEngine()

def dt(value: str) -> datetime:
    return datetime.fromisoformat(value.replace("Z", "+00:00")).astimezone(time

# Encode time windows as datetimes
nightfall  = TemporalInterval(start=dt("2025-01-01T00:00:00Z"), end=dt("2025-03
sandstorm  = TemporalInterval(start=dt("2025-03-15T00:00:00Z"), end=dt("2025-06
frostbite  = TemporalInterval(start=dt("2025-07-01T00:00:00Z"), end=dt("2025-09

relation_ns = engine.relation(nightfall, sandstorm)
relation_nf = engine.relation(nightfall, frostbite)

print("NIGHTFALL vs SANDSTORM:", relation_ns)
# IntervalRelation.OVERLAPS — both active simultaneously in mid-March 2025
# Warrants investigation for shared C2 infrastructure or coordination

print("NIGHTFALL vs FROSTBITE:", relation_nf)
# IntervalRelation.BEFORE — no temporal overlap; likely independent campaigns
```

The 13 Allen relations cover every possible temporal relationship:

RELATION	MEANING
`BEFORE`	A ends before B starts
`MEETS`	A ends exactly where B starts (no gap, no overlap)
`OVERLAPS`	A starts before B and ends during B
`STARTS`	A and B start together; A ends first
`DURING`	A is fully contained within B
`FINISHES`	A and B end together; A starts later
`EQUALS`	A and B are identical intervals
`AFTER`, `MET_BY`, `OVERLAPPED_BY`, `STARTED_BY`, `CONTAINS`, `FINISHED_BY`	Inverses of the above

An `OVERLAPS` or `EQUALS` result between two campaigns attributed to different actors is a signal worth flagging for analyst review — a temporal coincidence is a hypothesis, not a conclusion.

Step 8 — LLM-based graph reasoning

`GraphReasoner` routes freeform natural language queries through an LLM provider, using the graph as grounded context. Use it for exploratory questions that do not map cleanly to a predefined rule set:

```
from semantica.reasoning import GraphReasoner
from semantica.context import ContextGraph

# Initialise with any supported LLM provider
gr = GraphReasoner(provider="openai", model="gpt-4o-mini")

# Build a knowledge graph
graph = ContextGraph()
graph.add_node("apt29",          "ThreatActor",   "APT29 / NOBELIUM", country="
graph.add_node("cve-2025-3400",  "Vulnerability", "PAN-OS RCE",       cvss=9.8)
graph.add_node("nato_logistics", "Target",        "NATO Logistics Network")
graph.add_edge("apt29", "cve-2025-3400",  "exploits", weight=0.97)
graph.add_edge("apt29", "nato_logistics", "targets",  weight=0.88)

# GraphReasoner expects {"entities": [...], "relationships": [...]}
raw = graph.to_dict()
graph_data = {
    "entities":      raw.get("nodes", []),
    "relationships": raw.get("edges", []),
}

answer = gr.reason(
    graph=graph_data,
    query="Which threat actors pose the highest risk to NATO infrastructure, "
          "and what evidence in the graph supports that assessment?"
)
print(answer)
```

`GraphReasoner` is well suited for early-stage investigation — when the question is exploratory and you have not yet formalised inference rules. For reproducible, auditable decisions, use `Reasoner` or `DatalogReasoner` instead.

Step 9 — Explaining inferences in natural language

`ExplanationGenerator` translates any `InferenceResult` (from forward or backward chaining) into a human-readable explanation, a step-by-step `ReasoningPath`, and a `Justification` with supporting evidence:

```
from semantica.reasoning import Reasoner, Rule, RuleType, Explanati

# Run inference first
reasoner = Reasoner()
reasoner.add_fact("ThreatActor(APT29)")
reasoner.add_fact("Exploits(APT29, CVE-2025-3400)")
reasoner.add_fact("CriticalVuln(CVE-2025-3400)")

reasoner.add_rule(Rule(
    rule_id="r1", name="high_risk_actor",
    conditions=["ThreatActor(X)", "Exploits(X, Y)", "CriticalVuln(Y)"],
    conclusion="HighRiskActor(X)",
    confidence=0.92,
))

derived = reasoner.forward_chain()

# detail_level options: "simple", "detailed", "verbose"
gen = ExplanationGenerator(generate_nl=True, detail_level="detailed")

for result in derived:
    exp = gen.generate_explanation(result)
    print("Conclusion:  {}".format(exp.conclusion))
    print("Explanation: {}".format(exp.natural_language))
    print()

    # Step-by-step reasoning path
    path = gen.show_reasoning_path(result)
    print("Reasoning path ({} steps, confidence {:.0%}):".format(
        len(path.steps), path.total_confidence
    ))
    for step in path.steps:
        print("  [{}] {}".format(step.step_id, step.description))

    # Justification with full evidence list
    just = gen.justify_conclusion(result.conclusion, path)
    print("Justification: {}".format(just.explanation_text))
    print("Supporting evidence: {}".format(just.supporting_evidence))
```

```
Conclusion:  HighRiskActor(APT29)
Explanation: Given the premises: ThreatActor(APT29), Exploits(APT29, CVE-2025-3
             CriticalVuln(CVE-2025-3400), we conclude: HighRiskActor(APT29)
             using rule 'high_risk_actor'.
```

Three detail levels control explanation verbosity: `"simple"` gives a one-line summary, `"detailed"` lists premises and the rule name, and `"verbose"` produces a full confidence-annotated narrative.

Putting it together: a complete reasoning pipeline

A pipeline combining forward chaining, Datalog reachability, and natural language explanations for a threat intelligence graph:

```
from semantica.reasoning import (
    Reasoner, Rule, RuleType,
    DatalogReasoner, ExplanationGenerator,
)
from semantica.context import AgentContext, ContextGraph
from semantica.vector_store import VectorStore


def run_threat_reasoning(graph: ContextGraph) -> dict:
    """Apply inference rules to a threat intelligence graph."""

    # --- Forward chaining: derive actor classifications ---
    reasoner = Reasoner()

    for edge in graph.find_edges():
        src = edge.get("source", "")
        dst = edge.get("target", "")
        rel = edge.get("type", "related_to")
        if src and dst:
            reasoner.add_fact("{}({}, {})".format(rel.replace(" ", "_"), src, d

    for node in graph.find_nodes():
        name  = node.get("name", node.get("id", ""))
        ntype = node.get("type", "Entity")
        if name:
            reasoner.add_fact("{}({})".format(ntype.replace(" ", "_"), name))

    reasoner.add_rule(Rule(
        rule_id="r1", name="high_risk_actor",
        conditions=["ThreatActor(X)", "Exploits(X, CVE)", "CriticalVuln(CVE)"],
        conclusion="HighRiskActor(X)", confidence=0.92, priority=10,
    ))
    reasoner.add_rule(Rule(
        rule_id="r2", name="critical_target",
        conditions=["HighRiskActor(X)", "Targets(X, Z)"],
        conclusion="CriticalTarget(Z)", confidence=0.88, priority=8,
    ))
    reasoner.add_rule(Rule(
        rule_id="r3", name="supplier_elevation",
        conditions=["SuppliedExploits(A, B)", "HighRiskActor(B)"],
        conclusion="HighRiskSupplier(A)", confidence=0.85, priority=5,
    ))

    derived = reasoner.forward_chain()

    high_risk_actors    = [r for r in derived if "HighRiskActor"    in r.conclu
    critical_targets    = [r for r in derived if "CriticalTarget"   in r.conclu
    high_risk_suppliers = [r for r in derived if "HighRiskSupplier" in r.conclu

    # --- Backward chaining: verify a specific supplier hypothesis ---
    supplier_result = reasoner.backward_chain("HighRiskSupplier(DELTA-3)", max_

    # --- Datalog: transitive supply-chain reachability ---
    dl = DatalogReasoner()
    dl.load_from_graph(graph)
    dl.add_rule("reaches(X, Y) :- supplied(X, Y).")
    dl.add_rule("reaches(X, Y) :- supplied(X, Z), reaches(Z, Y).")
    dl.add_rule("sector_exposure(Actor, Sector) :- reaches(Actor, T), sector(T,
    dl.add_rule("sector_exposure(Actor, Sector) :- targets(Actor, T), sector(T,
    dl.derive_all()
    ci_exposures = dl.query("sector_exposure(?actor, critical_infrastructure)")

    # --- ExplanationGenerator: analyst-readable justifications ---
    gen = ExplanationGenerator(generate_nl=True, detail_level="detailed")
    explanations = {}
    for result in high_risk_actors:
        exp = gen.generate_explanation(result)
```

```python
        explanations[result.conclusion] = exp.natural_language

    return {
        "high_risk_actors":    [r.conclusion for r in high_risk_actors],
        "critical_targets":    [r.conclusion for r in critical_targets],
        "high_risk_suppliers": [r.conclusion for r in high_risk_suppliers],
        "delta3_flagged":      supplier_result is not None,
        "ci_exposed_actors":   [row["actor"] for row in ci_exposures],
        "total_derived":       len(derived),
        "explanations":        explanations,
    }


intel_graph = ContextGraph(advanced_analytics=True)
agent = AgentContext(
    vector_store=VectorStore(backend="faiss", dimension=768),
    knowledge_graph=intel_graph,
)
# ... populate via agent.store() or extraction pipeline ...

report = run_threat_reasoning(intel_graph)
print("Derived {} new facts".format(report["total_derived"]))
print("High-risk actors:  {}".format(report["high_risk_actors"]))
print("CI-exposed actors: {}".format(report["ci_exposed_actors"]))
print("DELTA-3 flagged:   {}".format(report["delta3_flagged"]))

for conclusion, text in report["explanations"].items():
    print("\n[{}]\n  {}".format(conclusion, text))
```

Domain examples

Defense — CTI/Threat   Security — SOC/Incident   Life Science — Clinical/Pharma   Banking — R

Attribution chains in threat intelligence require multi-hop confidence propagation: a TTP match raises the probability of actor attribution, corroborating ASN geolocation raises it further, and a known targeting pattern for the attributed sector raises it to actionable confidence. Each hop is a separate rule with its own confidence weight, and `InferenceResult` carries the propagated value through the chain.

```python
from semantica.reasoning import Reasoner, Rule, RuleType

reasoner = Reasoner()

# SIGINT-derived facts
reasoner.add_fact("C2Beacon(10.0.0.5, AS59796)")
reasoner.add_fact("ASN_Country(AS59796, Russia)")
reasoner.add_fact("TTP(T1566.001, APT29)")          # MITRE ATT&CK mapping
reasoner.add_fact("ObservedTTP(10.0.0.5, T1566.001)")
reasoner.add_fact("TargetSector(10.0.0.5, Aerospace)")

# Three-stage attribution ruleset with confidence ladder
reasoner.add_rule(Rule(
    rule_id="attr-1", name="ttp_match",
    conditions=["ObservedTTP(IP, TTP)", "TTP(TTP, Actor)"],
    conclusion="SuspectedActor(IP, Actor)",
    confidence=0.75, priority=10,
))
reasoner.add_rule(Rule(
    rule_id="attr-2", name="asn_corroboration",
    conditions=["SuspectedActor(IP, Actor)", "C2Beacon(IP, ASN)", "ASN_Country(
    conclusion="CorroboratedActor(IP, Actor, Country)",
    confidence=0.90, priority=5,
))
reasoner.add_rule(Rule(
    rule_id="attr-3", name="sector_confirmation",
    conditions=["CorroboratedActor(IP, Actor, Russia)", "TargetSector(IP, Aeros
    conclusion="HighConfidenceAttribution(IP, Actor)",
    confidence=0.95, priority=1,
))

derived = reasoner.forward_chain()
attributions = [r for r in derived if "HighConfidenceAttribution" in r.conclusi

for a in attributions:
    print("{:50s}  conf={:.0%}".format(a.conclusion, a.confidence))
    print("  Premises: {}".format(a.premises))

# HighConfidenceAttribution(10.0.0.5, APT29)    conf=95%
#   Premises: ['SuspectedActor(10.0.0.5, APT29)', 'CorroboratedActor(10.0.0.5,
```

Common Pitfalls

Overusing reasoning. Not every query needs inference. If you can answer your question with simple retrieval or graph traversal, reasoning adds unnecessary complexity. Use reasoning when you need to derive facts that aren’t explicitly stated.

Poor graph quality. Reasoning amplifies data quality issues. If your graph has inconsistent entity names, missing relationships, or incorrect facts, reasoning will propagate these errors. Clean your graph data before applying inference rules.

Treating inferred facts as verified facts. Reasoning conclusions are only as reliable as the rules and facts they’re based on. An inferred fact like “HighRiskActor(APT29)” reflects your rule logic, not ground truth. Always distinguish between observed facts and inferred conclusions.

Excessive rule complexity. Complex rules with many conditions are hard to debug and maintain. Start with simple rules and add complexity gradually. A rule with 10 conditions probably should be broken into smaller, more focused rules.

Recursive reasoning on large datasets. Recursive Datalog rules can generate exponential numbers of derived facts on large graphs. Monitor working memory size and add depth limits or termination conditions to prevent runaway inference.

Related Guides

- **Semantic Extraction** — extract the entities and relationships that populate the graph facts you reason over
- **GraphRAG** — retrieve graph-grounded context for LLM responses
- **Ontology Management** — generate OWL ontologies to give your rules formal semantics
- **Decision Intelligence** — record and trace inferred decisions through the full causal chain
- **Context Graphs** — the knowledge graph that reasoning operates over
- **MCP Server** — expose `run_reasoning` as a tool for Claude and other agents

‹ Ontology Management
Export & Serialization ›

Powered by mintlify